Analytics & Visualization

Day 3 · The Haunted Snack Bar, then five years of seasonal data

Sobey High School Co-Op · Business Analytics & AI · Friday, October 30, 2026

TODAY

What we're doing

Recap	three Day 2 mistakes and the one-line fix for each
Notebook 1 · The Haunted Snack Bar	type along: variables, lists, for, if / elif / else, functions, while, dictionaries, a DataFrame
The notebook	the tools you just used, the pipeline from question to decision, and why we draw charts
Notebook 2 · The Seasonal Workbook	five years of Google Trends for eight seasonal products, plus Halloween spending: seven chart types, one question each
Season maths	peak week, stock-up week, season length: three numbers every seasonal business needs
Pitch the season	teams of 2 or 3, one lane each: a stocking decision backed by 2 or 3 charts, title + 3 slides
Presentations	3 minutes per team, in season order, then the takeaway

Everyone runs their own copy of both notebooks. The pitch is in teams.

BY 1 PM

Five things you can do by the end of today



Write eight Python ideas

a variable, a list, a loop, an if, a function, a while, a dictionary, a DataFrame: typed by you, not pasted



Load and summarize real data

2,096 weeks of Canadian searches in a table you control with code



Pick the right chart

line, bar, grouped bar, histogram, box plot, scatter, heatmap: one question each



Spot a misleading chart

and fix it: a zero baseline, a title that states the finding, a source line, a limit said out loud



Make a seasonal decision

"We built a program that analyzes and visualizes data to support a business decision."

Three mistakes from last week, and the one-line fix for each

FORGOT THE f

Mistake: print("{school} has {students}") printed the braces, no error.

```
school = "Sobey"
students = 20
print(f"{school} has {students} students")
```

```
Sobey has 20 students
```

int() CAN'T READ "seventeen"

Mistake: ValueError. Fix: a while loop that asks again.

```
answer = input("How old? ")
while not answer.isdigit():
    print("Type a number, like 17")
    answer = input("How old? ")
age = int(answer)
print(f"Next year: {age + 1}")
```

```
How old? seventeen
Type a number, like 17
How old? 17
Next year: 18
```

= STORES, == COMPARES

Mistake: if score = 80 (SyntaxError) and a body that wasn't indented.

```
score = 72
if score >= 80:
    print("A")
elif score >= 70:
    print("B")
else:
    print("C")
```

```
B
```

Habit: read the last line of an error first, then check that the output is what you meant, not only that it ran.

01

The Haunted Snack Bar: eight cells, typed by you

Eight words you'll use all day, in the order you'll type them

1	**Variable**	a named box that holds one value
2	**List**	many values in order, inside [ ]
3	**for loop**	do the same thing to every item in a list
4	**if / elif / else**	the program chooses between paths
5	**Function**	a recipe with a name: give it ingredients, it returns an answer
6	**while loop**	keep going as long as a condition is true
7	**Dictionary**	look values up by name, inside { }
8	**DataFrame**	a whole table pandas can sort, filter, group and chart

A variable is a named box: put a value in with =, use the name later

```
candy = "Reese's Cups"
price = 1.75
stock = 30
sold_out = False

print(f"{candy} costs ${price} and we have {stock} left")
print("Sold out?", sold_out)
```

```
Reese's Cups costs $1.75 and we have 30 left
Sold out? False
```

Four boxes: candy holds text (quotes), price a decimal, stock a whole number, sold_out a True/False. The f before the quotes lets {candy} and {price} drop their values into the sentence.

WHAT YOU SEE

One real sentence about the snack bar, built from four boxes.

Change one thing: stock = 0 and sold_out = True; then your favourite candy.

A list holds many values in order; Python counts from 0

```
candies = ["M&M's", "Sour Patch Kids", "Skittles", "Reese's Cups", "Candy Corn"]

print(candies)
print("First on the menu:", candies[0])
print("How many kinds:", len(candies))

candies.append("Butterfinger")
print(candies)
```

```
["M&M's", 'Sour Patch Kids', 'Skittles', "Reese's Cups", 'Candy Corn']
First on the menu: M&M's
How many kinds: 5
["M&M's", 'Sour Patch Kids', 'Skittles', "Reese's Cups", 'Candy Corn', 'Butterfinger']
```

Square brackets make a list. candies[0] is the first item because Python starts at 0. len counts the items; append adds one to the end.

WHAT YOU SEE

The list printed twice: five candies, then six after append.

Change one thing: candies[0] → candies[-1] (the last one); append your favourite instead.

A for loop does the same thing to every item in the list

```
for candy in candies:
    print("🎃", candy, "is on the menu")

print("That's", len(candies), "kinds of candy")
```

```
🎃 M&M's is on the menu
🎃 Sour Patch Kids is on the menu
🎃 Skittles is on the menu
🎃 Reese's Cups is on the menu
🎃 Candy Corn is on the menu
🎃 Butterfinger is on the menu
That's 6 kinds of candy
```

for candy in candies: runs the indented line once per item, with candy holding the next one each time. The un-indented print runs once, after the loop.

WHAT YOU SEE

One line per candy, then the count.

Change one thing: Add a candy to the list in cell 2 and run both: one more line with no new code. Then 🎃 → 👻.

if, elif and else let the program choose; exactly one path runs

```
stock = 9

if stock == 0:
    print("SOLD OUT 💀")
elif stock < 10:
    print("Running low 👻 only", stock, "left")
else:
    print("Plenty 🎃", stock, "left")
```

```
Running low 👻 only 9 left
```

Python checks the questions from the top and runs the first one that is true. == asks 'is it equal?'; a single = puts a value in a box. 9 is not 0, but it is under 10, so the middle path runs.

WHAT YOU SEE

One message, chosen by the number.

Change one thing: stock = 0, then stock = 40, then change the 10 to 20.

A function wraps the check so you can run it on any candy in one line

```
def shelf_check(candy, stock):
    if stock == 0:
        return candy + ": SOLD OUT 💀"
    elif stock < 10:
        return candy + ": running low 👻"
    else:
        return candy + ": plenty 🎃"

print(shelf_check("Candy Corn", 60))
print(shelf_check("Butterfinger", 9))
print(shelf_check("Hershey's Mini Bars", 0))
```

```
Candy Corn: plenty 🎃
Butterfinger: running low 👻
Hershey's Mini Bars: SOLD OUT 💀
```

def makes a recipe called shelf_check that needs two ingredients. Inside is cell 4's if / elif / else, using whatever stock was handed in. return sends the answer back. Three calls, three answers, one recipe.

WHAT YOU SEE

Three candies checked with one function.

Change one thing: Add a fourth print with your own candy; then change < 10 to < 50.

A while loop keeps going as long as the condition is true

```
stock = 5

while stock > 0:
    stock = stock - 1
    print("Sold one Reese's Cups!", stock, "left")

print("SOLD OUT 💀 time to restock")
```

```
Sold one Reese's Cups! 4 left
Sold one Reese's Cups! 3 left
Sold one Reese's Cups! 2 left
Sold one Reese's Cups! 1 left
Sold one Reese's Cups! 0 left
SOLD OUT 💀 time to restock
```

A for loop runs once per item. A while loop runs while stock > 0 is true, so something inside has to change: stock = stock - 1 takes one off the shelf each time round. When it hits 0 the loop stops and the last line runs.

WHAT YOU SEE

Five sales, then sold out.

Change one thing: stock = 10 and sell two at a time (stock - 2); then stock = 3.

A dictionary looks a value up by its name: a price list

```
prices = {"M&M's": 1.50, "Skittles": 1.25, "Reese's Cups": 1.75, "Candy Corn": 0.75}

print("Skittles cost $", prices["Skittles"])
prices["Hot Tamales"] = 1.00

order = ["Skittles", "Skittles", "Reese's Cups"]
total = 0
for item in order:
    total = total + prices[item]

print(f"Your order costs ${total:.2f}")
```

```
Skittles cost $ 1.25
Your order costs $4.25
```

Curly brackets; name, colon, value. prices["Skittles"] looks up by name. A new name-and-price pair is added with one line. Then cell 3's loop and cell 6's add-up work together: for every item in the order, look up its price and add it on.

WHAT YOU SEE

A price lookup, then an order total with two decimals.

Change one thing: Add "Candy Corn" to the order; raise the price of Reese's Cups.

pandas reads the whole file into one variable: a table with named columns

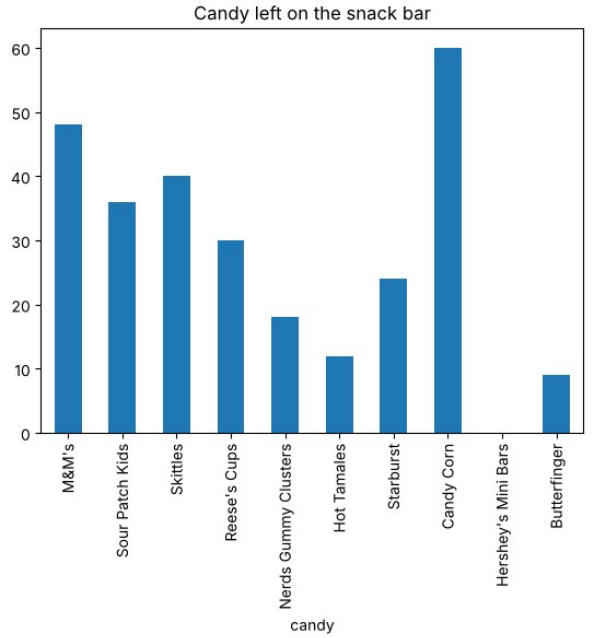
```
import pandas as pd

snacks = pd.read_csv("Snack Bar Data.csv")

snacks["value"] = snacks["price"] * snacks["stock"]
print("Candy on the shelf is worth $", snacks["value"].sum())

snacks.plot.bar(x="candy", y="stock", title="Candy left on the snack bar", legend=False)
snacks
```

```
Candy on the shelf is worth $ 356.0
   rank                 candy       type  price  stock  value
0     1                 M&M's  chocolate   1.50     48   72.0
1     2       Sour Patch Kids       sour   1.25     36   45.0
2     3              Skittles     fruity   1.25     40   50.0
3     4          Reese's Cups  chocolate   1.75     30   52.5
4     5  Nerds Gummy Clusters     fruity   2.00     18   36.0
5     6           Hot Tamales   cinnamon   1.00     12   12.0
6     7             Starburst     fruity   1.25     24   30.0
7     8            Candy Corn    classic   0.75     60   45.0
8     9   Hershey's Mini Bars  chocolate   1.50      0    0.0
9    10          Butterfinger  chocolate   1.50      9   13.5
```



read_csv turns the uploaded file into a DataFrame. A column times a column makes a new column for every row at once, no loop. .sum() adds a column up. One line draws the chart; a variable on its own as the last line shows the table.

Change one thing: y="stock" → y="value"; .sum() → .max().

WHAT YOU SEE

The 10 most popular Halloween candies (a real 2026 ranking), the shelf's value, and your first chart.

02

The notebook: what you just used, and why charts

THE TOOLS

Five tools, one job each, and where you already used them

Python	the language: the instructions you typed in cells 1 to 7
Google Colab	runs Python in your browser; the Files panel is where the CSV went
pandas	tables you control with code: read_csv, a new column, .sum() in cell 8
matplotlib	draws any chart, piece by piece: cell 8's bar chart used it behind the scenes
seaborn	statistical charts in one line: box plots and heatmaps, coming up in Notebook 2

Import once, print the versions, then every chart uses the same house style

```
import pandas as pd
import numpy as np
import matplotlib
import matplotlib.pyplot as plt
import seaborn as sns
import calendar, io, os, hashlib
print("pandas", pd.__version__, "· matplotlib", matplotlib.__version__, "· seaborn", sns.__version__)
```

```
pandas 3.0.5 · matplotlib 3.11.2 · seaborn 0.13.2
```

Versions shown: the run that produced the answer sheet.

WHY PRINT VERSIONS?

Different versions can draw a chart differently. Printing them makes your run checkable.

THE HOUSE STYLE

One ready-made cell sets colours, fonts and a finish() helper: a title that states the finding, a source line, a saved PNG.

Six steps: question, load, clean, analyze, visualize, decide



Question	Load	Clean	Analyze	Visualize	Decide
When should a store stock up for its season, and for how long?	2,096 weeks of Canadian searches, eight lanes; 22 years of Halloween spending	text dates become real dates; partial years get flagged, never hidden	groupby by month and year; peak week, stock-up week, season length	seven chart types, one question each	stock date, ad start, markdown date, and how to check it

Cell 8 was the whole pipeline in miniature: a question (what is on the shelf worth?), load, a new column, a sum, a chart. Notebook 2 is the same shape with 2,096 rows.

pandas reads 2,096 weeks of real searches; a fingerprint proves the file

```python
wide = pd.read_csv(io.StringIO(BUILTIN_TRENDS))                        # one column per lane
trends = wide.melt(id_vars="week", var_name="lane", value_name="interest")   # one row per week per lane
source = "built-in copy"
if os.path.exists("Seasonal Trends Canada.csv"):                        # your uploaded file wins, if there is one
    trends = pd.read_csv("Seasonal Trends Canada.csv")[["week", "lane", "interest"]]
    source = "your uploaded Seasonal Trends Canada.csv"
spending = pd.read_csv(io.StringIO(BUILTIN_SPENDING))

print("Trends:", trends.shape[0], "rows ×", trends.shape[1], "columns, from the", source)
print("Spending:", spending.shape[0], "rows ×", spending.shape[1], "columns")
trends.head()
```

```
Trends: 2096 rows × 3 columns, from the built-in copy
Spending: 22 rows × 16 columns
         week               lane  interest
0  2021-10-24  halloween costume       100
1  2021-10-31  halloween costume        26
2  2021-11-07  halloween costume         3
3  2021-11-14  halloween costume         2
4  2021-11-21  halloween costume         1
```

TWO TABLES

Trends: 2,096 rows (week, lane, interest) and NRF spending (22 years). melt turns one column per lane into one row per week per lane, the shape groupby wants.

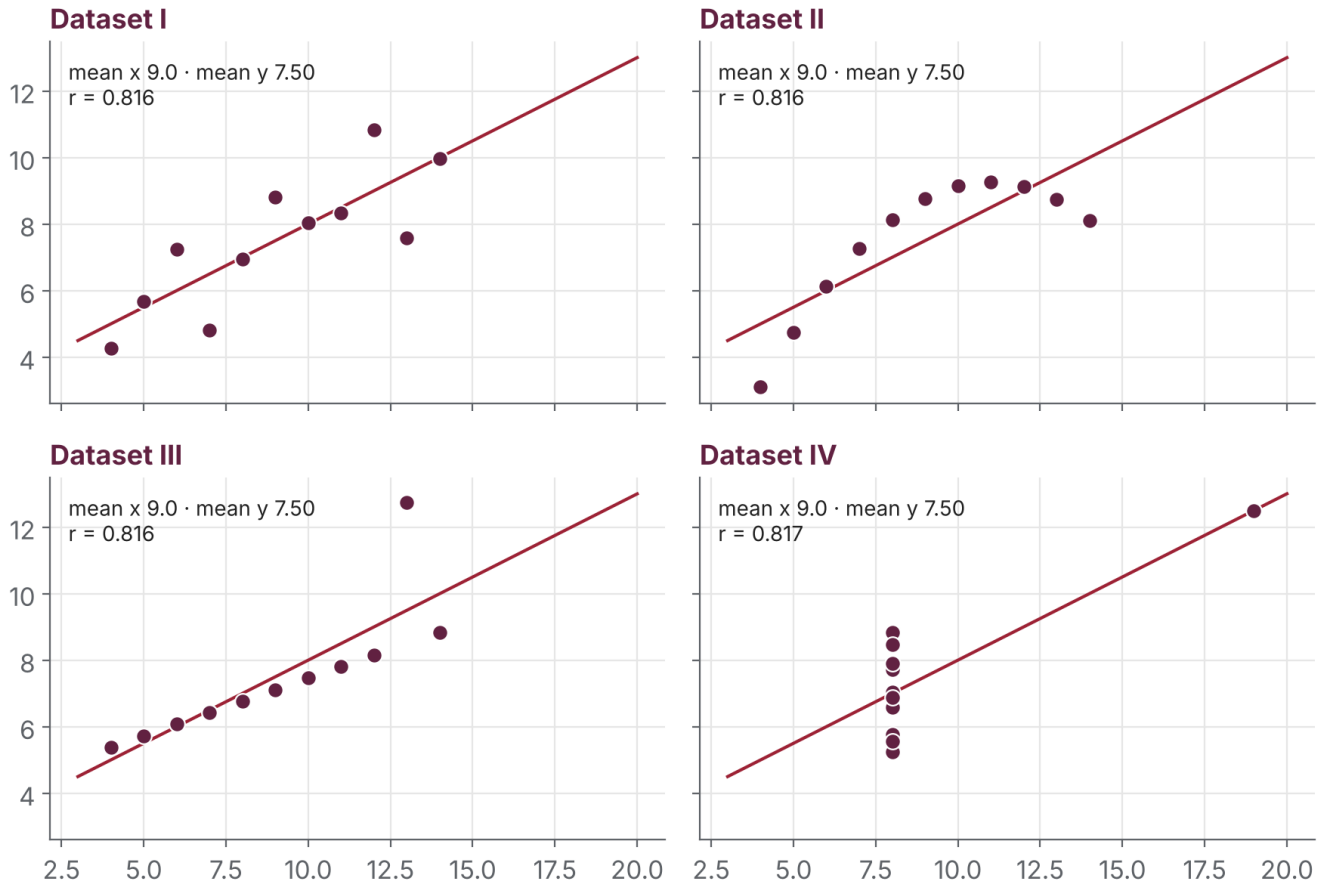
WHAT A SCORE MEANS

100 = that lane's busiest week; 50 = half as busy. Relative interest, not a count, not sales.

Feeds: every chart in the gallery and your lane in section 6.

Source: Google Trends, Canada, web search, weekly, Oct 24, 2021 to Oct 25, 2026; relative interest 0 to 100 per lane; read Oct 29, 2026

Four datasets, identical averages: only the charts show the difference



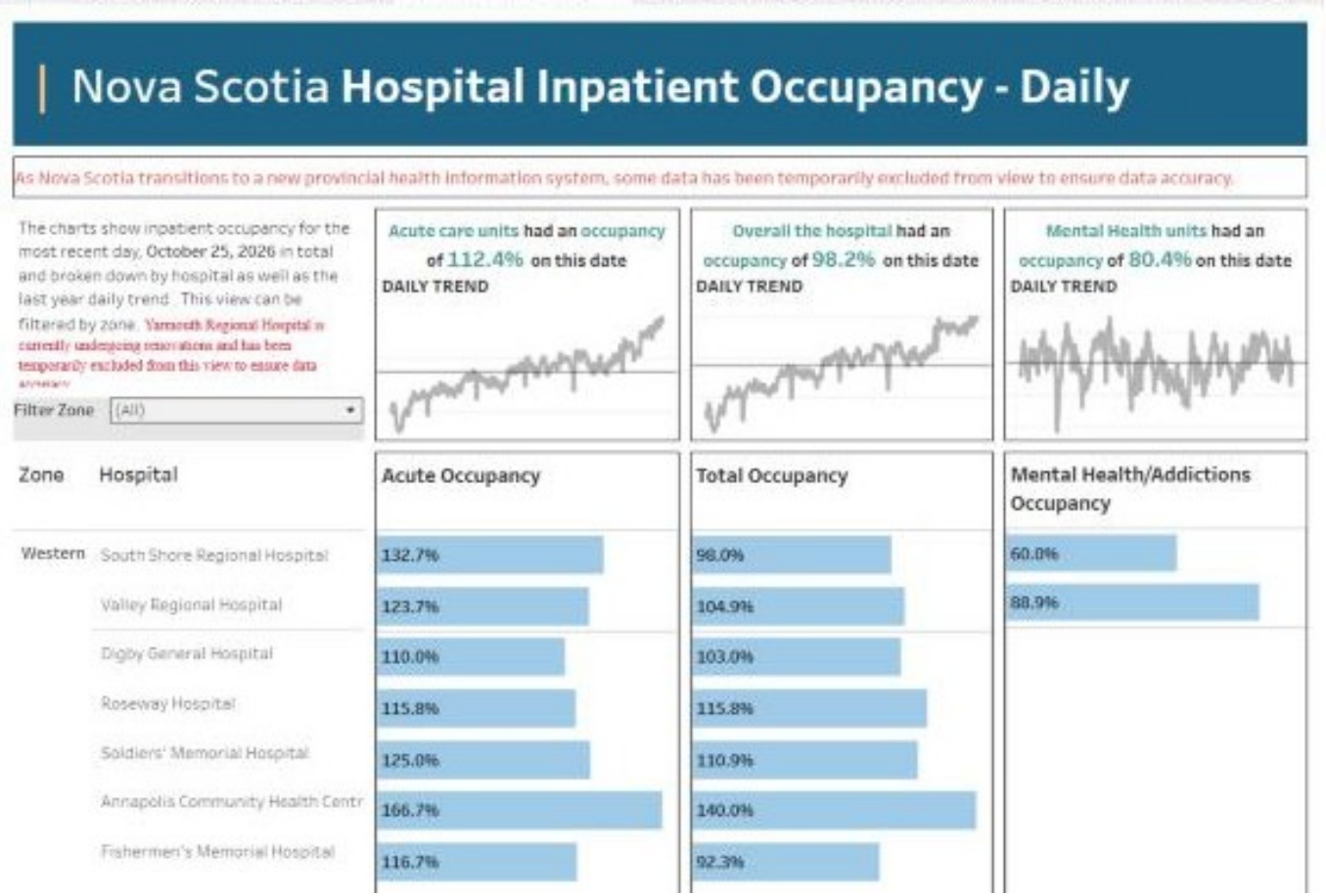
THE SAME FOR ALL FOUR

mean of x	**9.0**
mean of y	**7.50**
correlation r	**0.816**
points	**11 each**

So: always look at the data, not only its averages.

Source: F. J. Anscombe (1973), "Graphs in Statistical Analysis", The American Statistician 27(1); data as published in seaborn-data (anscombe.csv). Charts drawn by us.

Nova Scotia Health puts hospital occupancy on a public dashboard



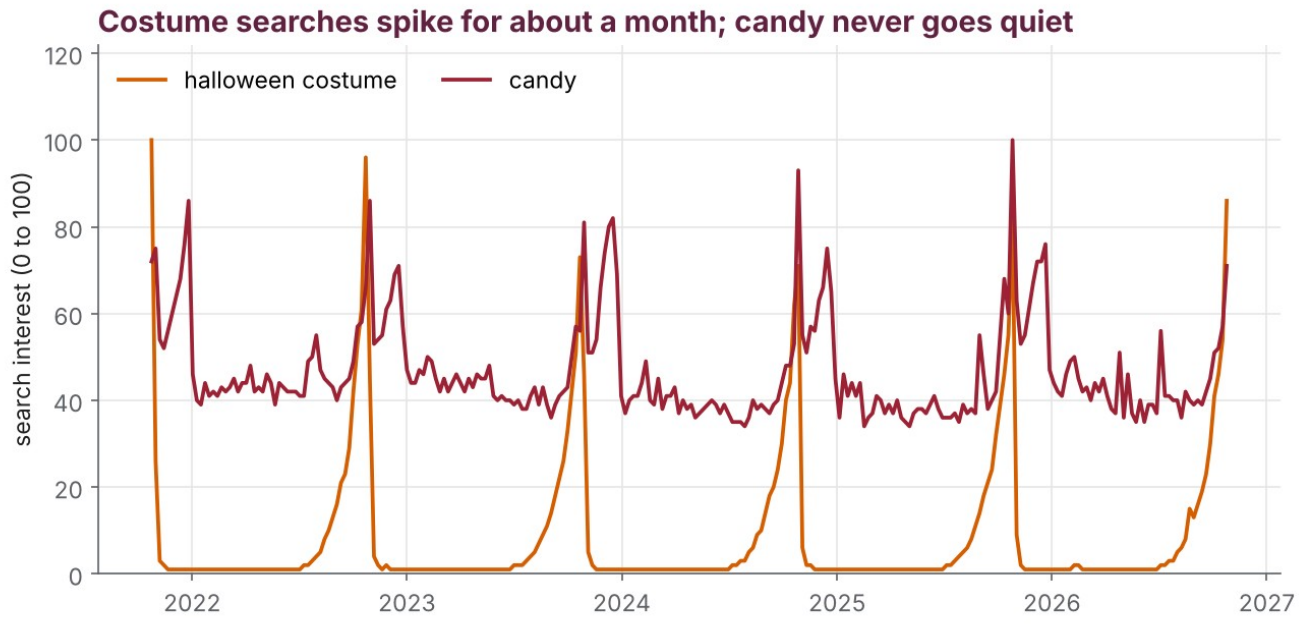
1. **Headline numbers**
   Acute care units at 112.4% occupancy on Oct 25, 2026
2. **Trend lines**
   a small daily trend under each number
3. **Bars per hospital**
   grouped by health zone, filterable
4. **An honest limit**
   the banner says some data is left out for now

Source: Nova Scotia Health, Action for Health – Public Reporting, "Hospital Inpatient Occupancy – Daily" (Tableau Public), data for Oct 25, 2026; screenshot Oct 27, 2026.

03

The chart gallery: one question per chart

Costume searches spike for about a month; candy never goes quiet



Source: Google Trends, Canada, web search, weekly, Oct 2021 to Oct 2026 (relative interest, 0 to 100 per lane)

THE QUESTION

When does Halloween start on Google?

MATPLOTLIB

```
ax.plot(one["week"], one["interest"],
        color=COLOURS[lane], label=lane)
```

WHAT WE CAN INFER

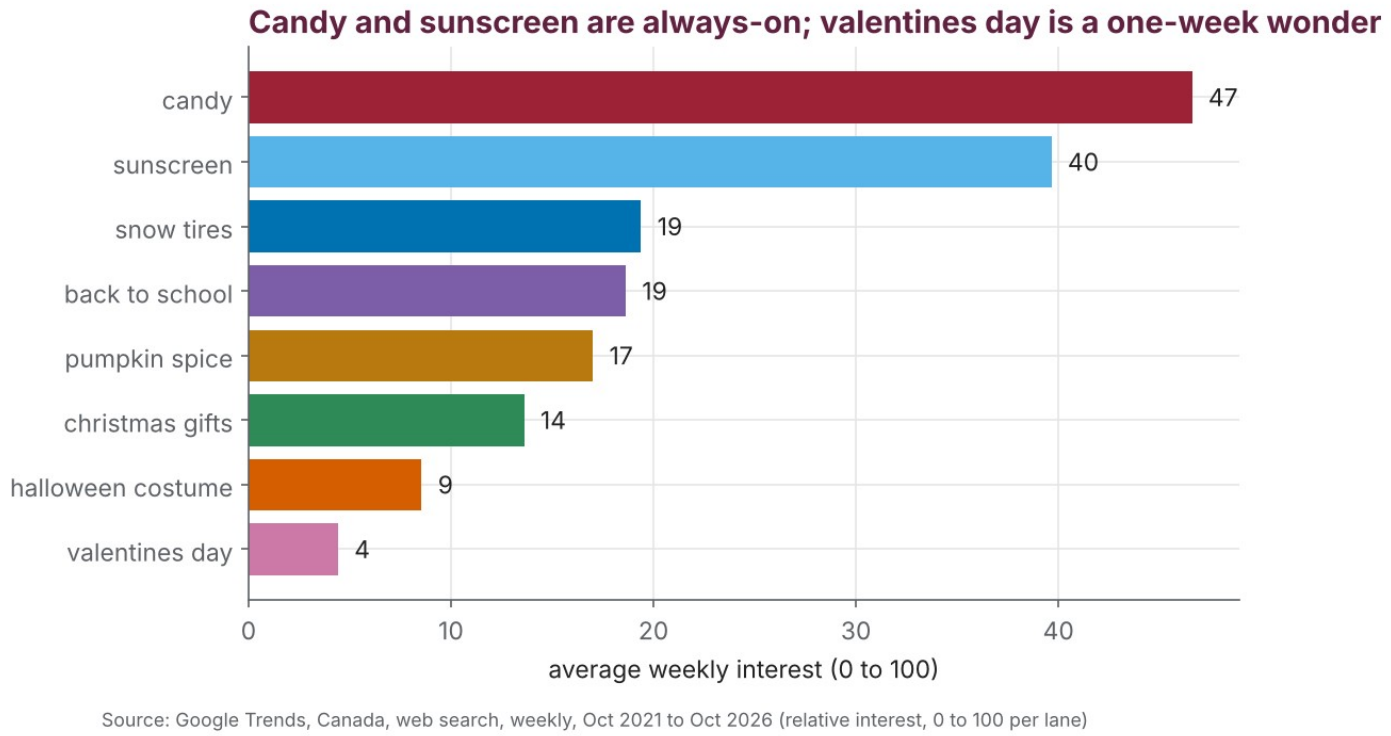
Costume interest is near zero for ten months, climbs through September and explodes in the last two weeks of October, the same shape every year. Candy sits between 35 and 60 with two bumps: Halloween and Christmas.

THE LIMIT

Relative interest, not sales: we know when people search, not how many or what they bought.

Source: Google Trends, Canada, web search, weekly, Oct 24, 2021 to Oct 25, 2026; relative interest 0 to 100 per lane; read Oct 29, 2026

Candy and sunscreen are always-on; valentines day is a one-week wonder



Source: Google Trends, Canada, web search, weekly, Oct 2021 to Oct 2026 (relative interest, 0 to 100 per lane)

THE QUESTION

Which lanes are always-on and which are spikes?

MATPLOTLIB

```
means = trends.groupby("lane")
["interest"].mean()
ax.barh(means.index, means.values)
```

WHAT WE CAN INFER

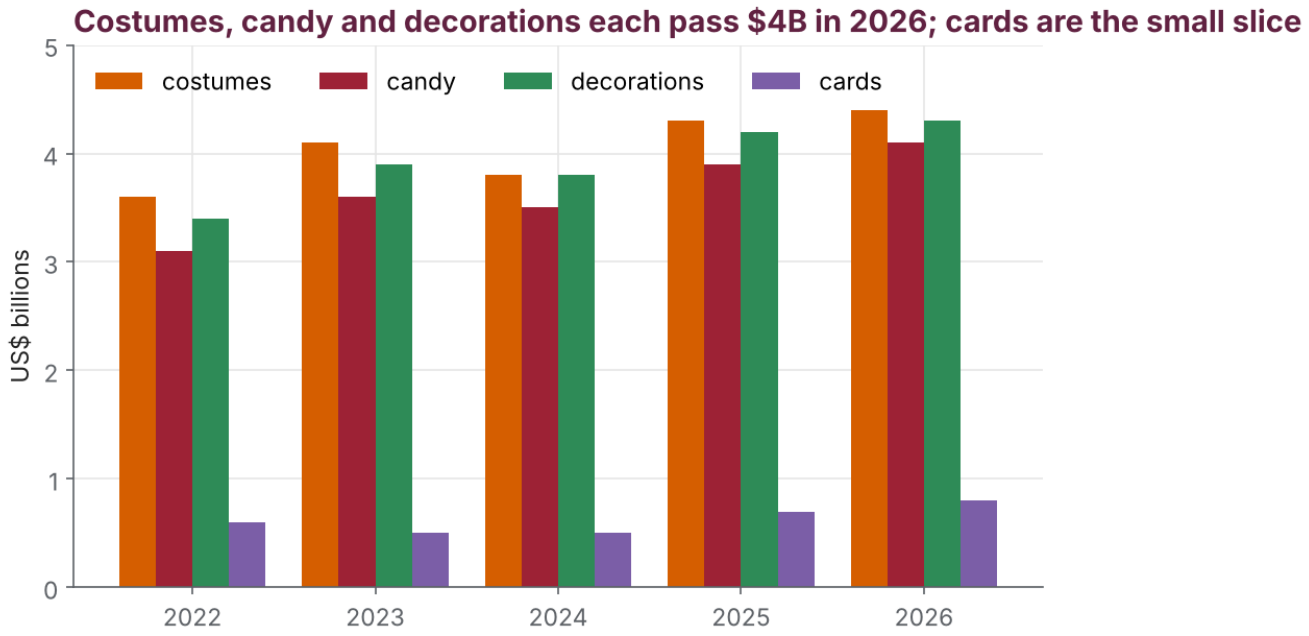
The average tells you how spread out a season is. A store in a spike lane lives or dies on timing; an always-on lane needs steady stock and a plan for the bumps.

THE LIMIT

An average hides when. The heatmap answers that.

Source: Google Trends, Canada, web search, weekly, Oct 24, 2021 to Oct 25, 2026; relative interest 0 to 100 per lane; read Oct 29, 2026

Costumes, candy and decorations each pass $4B; cards are the small slice



Source: NRF / Prosper Insights & Analytics annual Halloween survey (US consumers, US dollars; 2026 is the September forecast)

THE QUESTION

Where does the Halloween dollar go?

PANDAS + MATPLOTLIB

```
recent.plot.bar(ax=ax, width=0.8)   # one
bar per
# category, grouped by year
```

WHAT WE CAN INFER

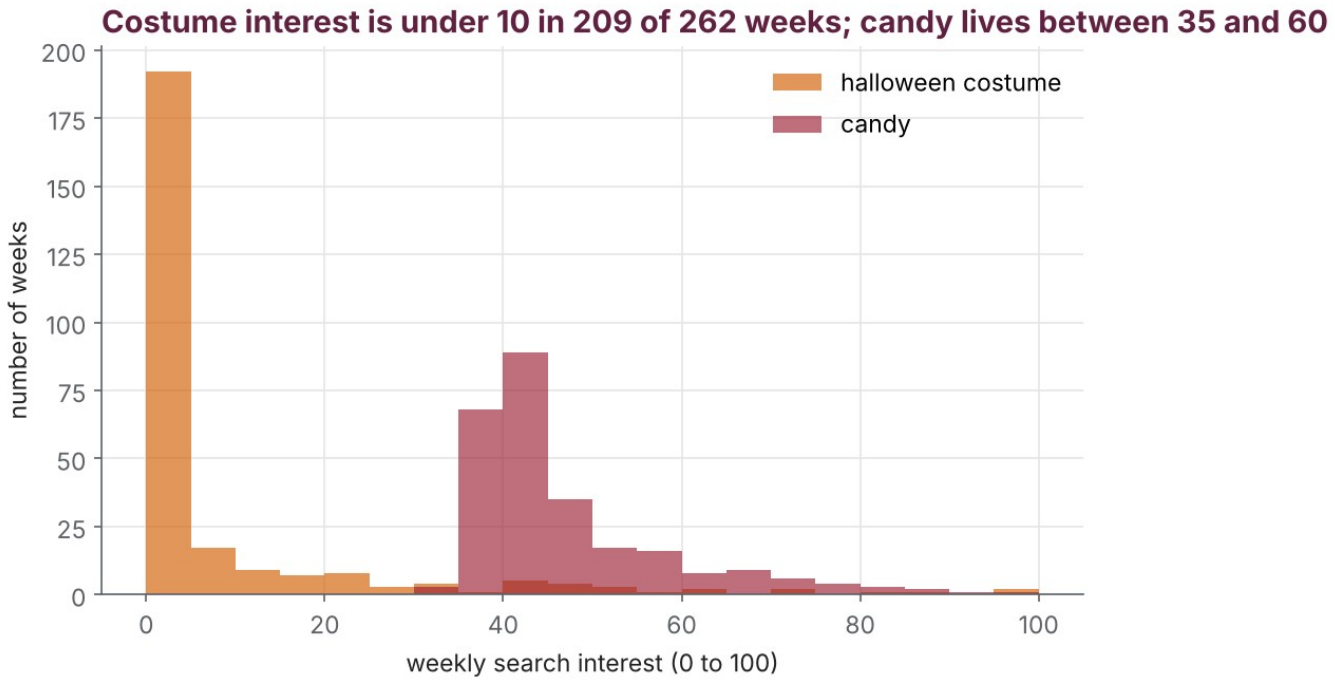
Three categories of roughly equal size, all growing, with a dip in 2024 (the $11.6B year). 2026: costumes $4.4B, candy $4.1B, decorations $4.3B, cards $0.8B.

THE LIMIT

US shoppers, US dollars, a survey forecast each September. Canada will rhyme with it; we do not have the Canadian number.

Source: NRF / Prosper Insights & Analytics annual Halloween consumer survey (nrf.com Halloween Data Center); US consumers, US dollars; 2026 is the September forecast

Costume interest is under 10 in 209 of 262 weeks; candy stays 35 to 60



Source: Google Trends, Canada, web search, weekly, Oct 2021 to Oct 2026 (relative interest, 0 to 100 per lane)

THE QUESTION

How many weeks a year are actually busy?

MATPLOTLIB

```
ax.hist(one["interest"], bins=range(0, 105,
5),
        alpha=0.65, label=lane)
```

WHAT WE CAN INFER

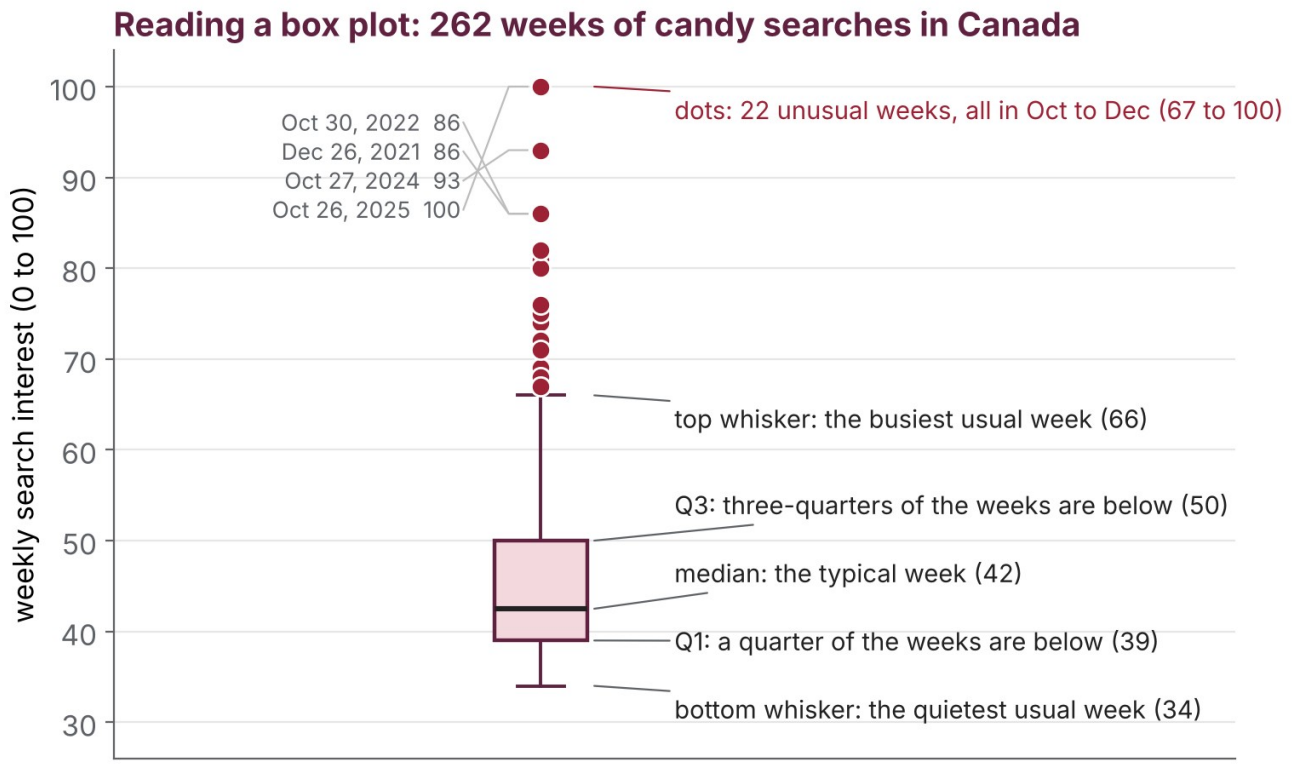
The shape of a season. A tall bar at the left plus a long tail is a spike business; a hump in the middle is a steady one. Costume scores above 50 in only 12 weeks out of 262.

THE LIMIT

Bins too narrow turn spiky and the shape disappears; bins too wide hide the tail. We used 5.

Source: Google Trends, Canada, web search, weekly, Oct 24, 2021 to Oct 25, 2026; relative interest 0 to 100 per lane; read Oct 29, 2026

How to read a box plot: the middle half in a box, the median as a line



Source: Google Trends, Canada, web search, weekly, Oct 2021 to Oct 2026 (relative interest, 0 to 100 per lane)

Box

the middle half of the weeks

Line in the box

the median: the typical week

Whiskers

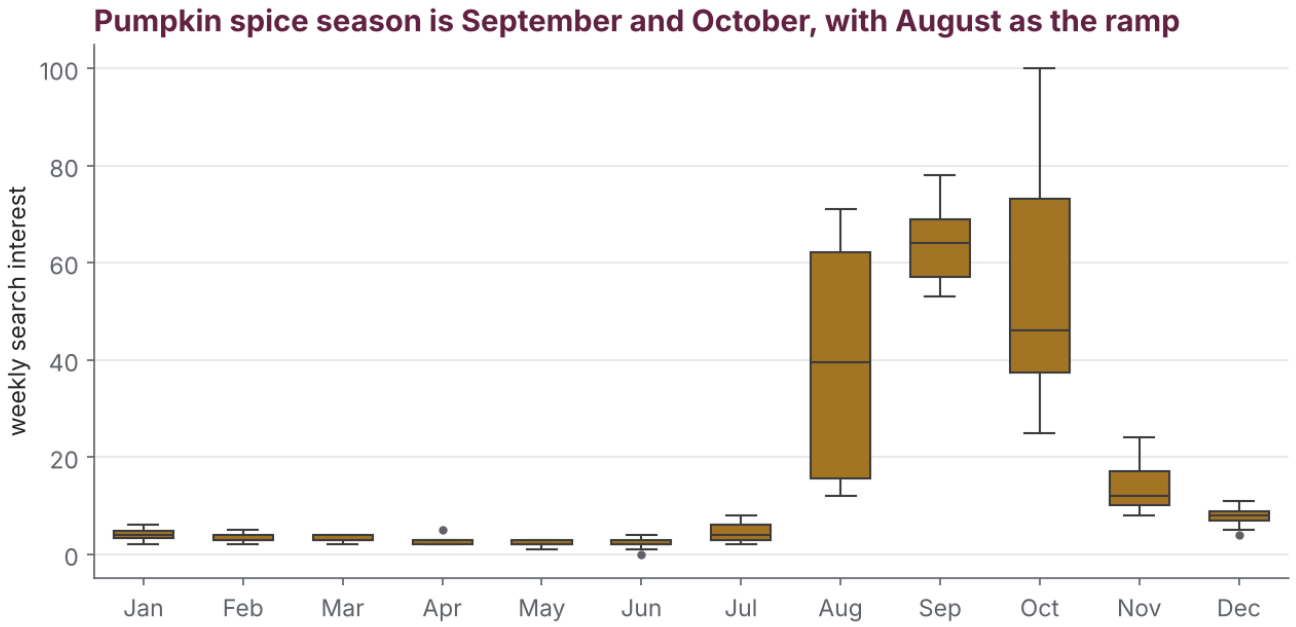
the rest of the usual weeks

Dots

unusual weeks, shown one by one: here, the season

Source: Google Trends, Canada, web search, weekly, Oct 24, 2021 to Oct 25, 2026; relative interest 0 to 100 per lane; read Oct 29, 2026

Pumpkin spice season is September and October, with August as the ramp



Source: Google Trends, Canada, web search, weekly, Oct 2021 to Oct 2026 (relative interest, 0 to 100 per lane)

THE QUESTION

Which month, and how sure are we?

SEABORN

```
sns.boxplot(data=one, x="month_name",
            y="interest", order=MONTHS)
```

WHAT WE CAN INFER

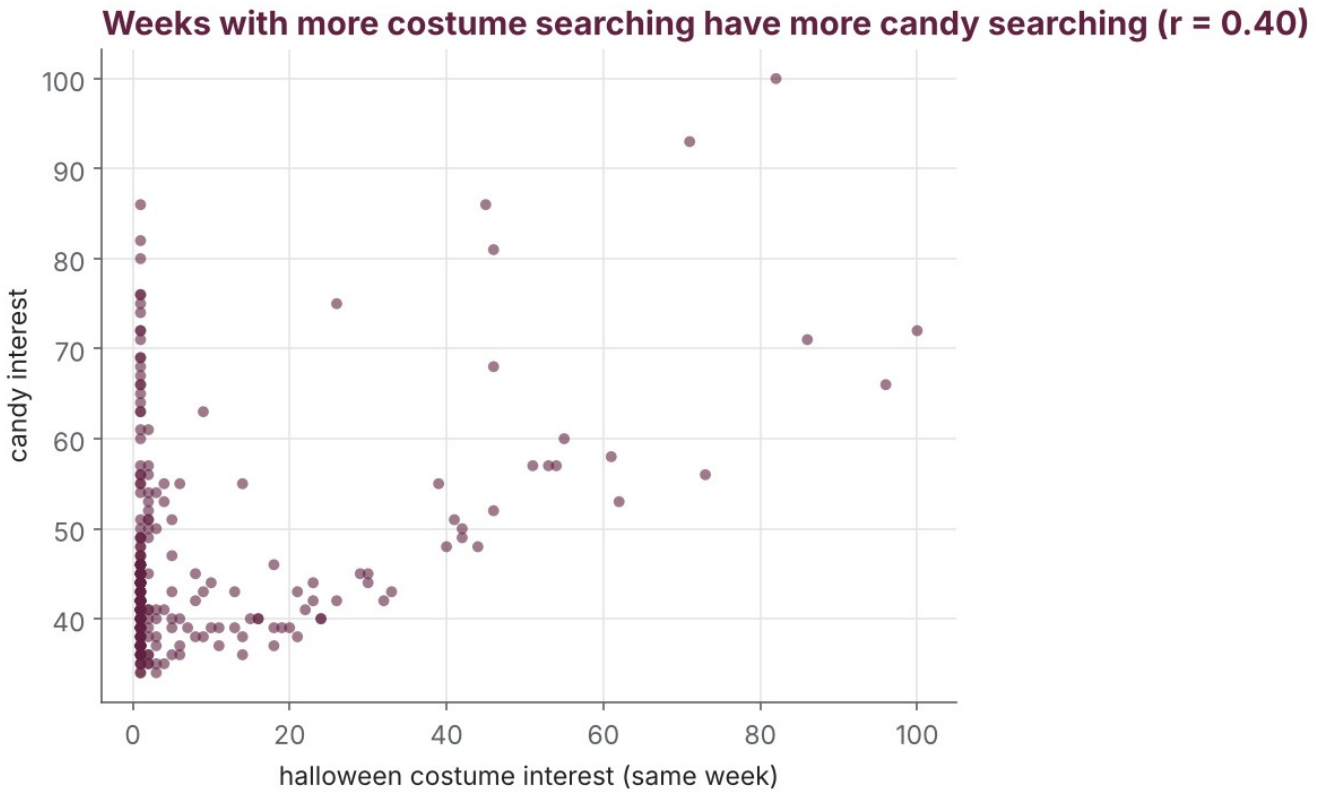
August's box is short and low but its whisker is tall: most August weeks are quiet, a couple are already busy. That whisker is the stock-up week. September and October boxes sit high: the season.

THE LIMIT

Five Augusts is a small sample; one strange year moves the box.

Source: Google Trends, Canada, web search, weekly, Oct 24, 2021 to Oct 25, 2026; relative interest 0 to 100 per lane; read Oct 29, 2026

Weeks with more costume searching have more candy searching (r = 0.40)



Source: Google Trends, Canada, web search, weekly, Oct 2021 to Oct 2026 (relative interest, 0 to 100 per lane)

THE QUESTION

Do costume and candy searches move together?

PANDAS + MATPLOTLIB

```
ax.scatter(pair["halloween costume"],
pair["candy"])
r = pair["halloween
costume"].corr(pair["candy"])
```

WHAT WE CAN INFER

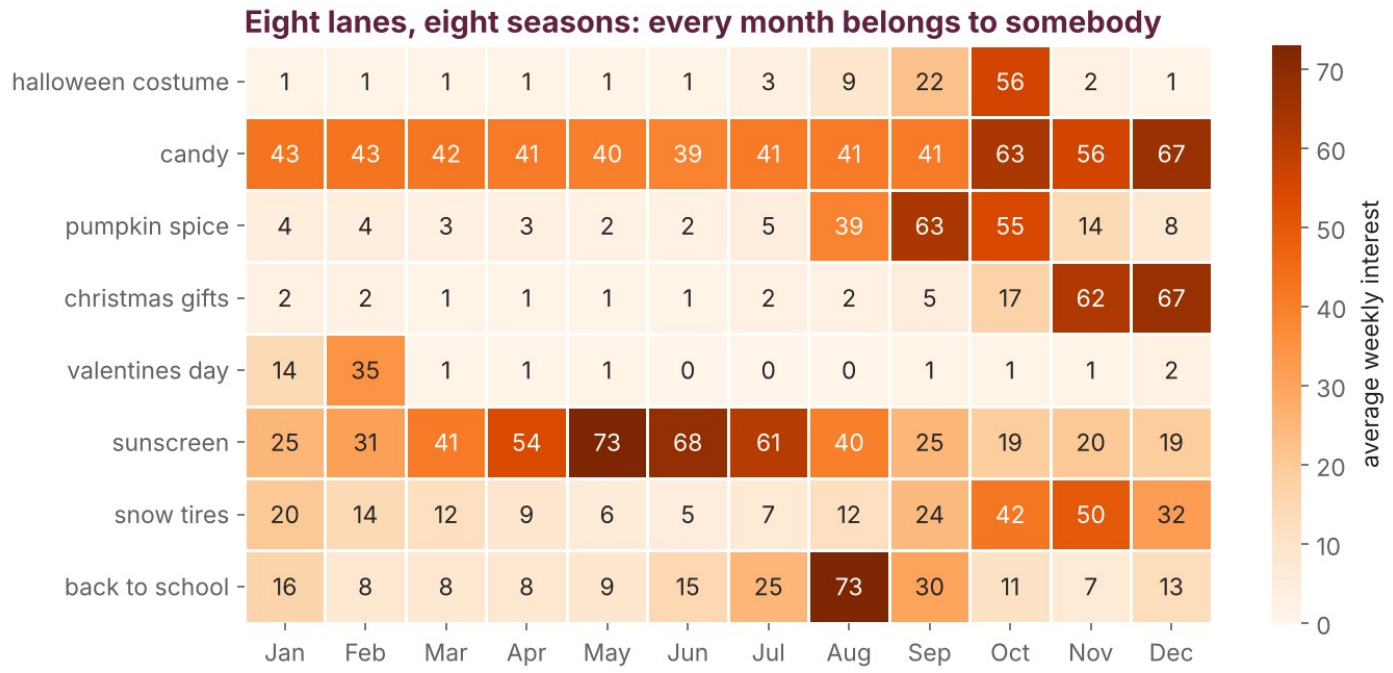
The dots lean up and to the right: r = 0.40, a moderate link. Most weeks are quiet for both; the busy costume weeks are only some of candy's busy weeks, because candy has Christmas too.

THE LIMIT

Not causation. Neither causes the other; the calendar causes both. Say it on your slide if you use a scatter.

Source: Google Trends, Canada, web search, weekly, Oct 24, 2021 to Oct 25, 2026; relative interest 0 to 100 per lane; read Oct 29, 2026

Eight lanes, eight seasons: every month belongs to somebody



Source: Google Trends, Canada, web search, weekly, Oct 2021 to Oct 2026 (relative interest, 0 to 100 per lane)

THE QUESTION

What does the whole year look like, for every lane at once?

SEABORN

```
sns.heatmap(calendar_table, annot=True,
            fmt=".0f", cmap="Oranges")
```

WHAT WE CAN INFER

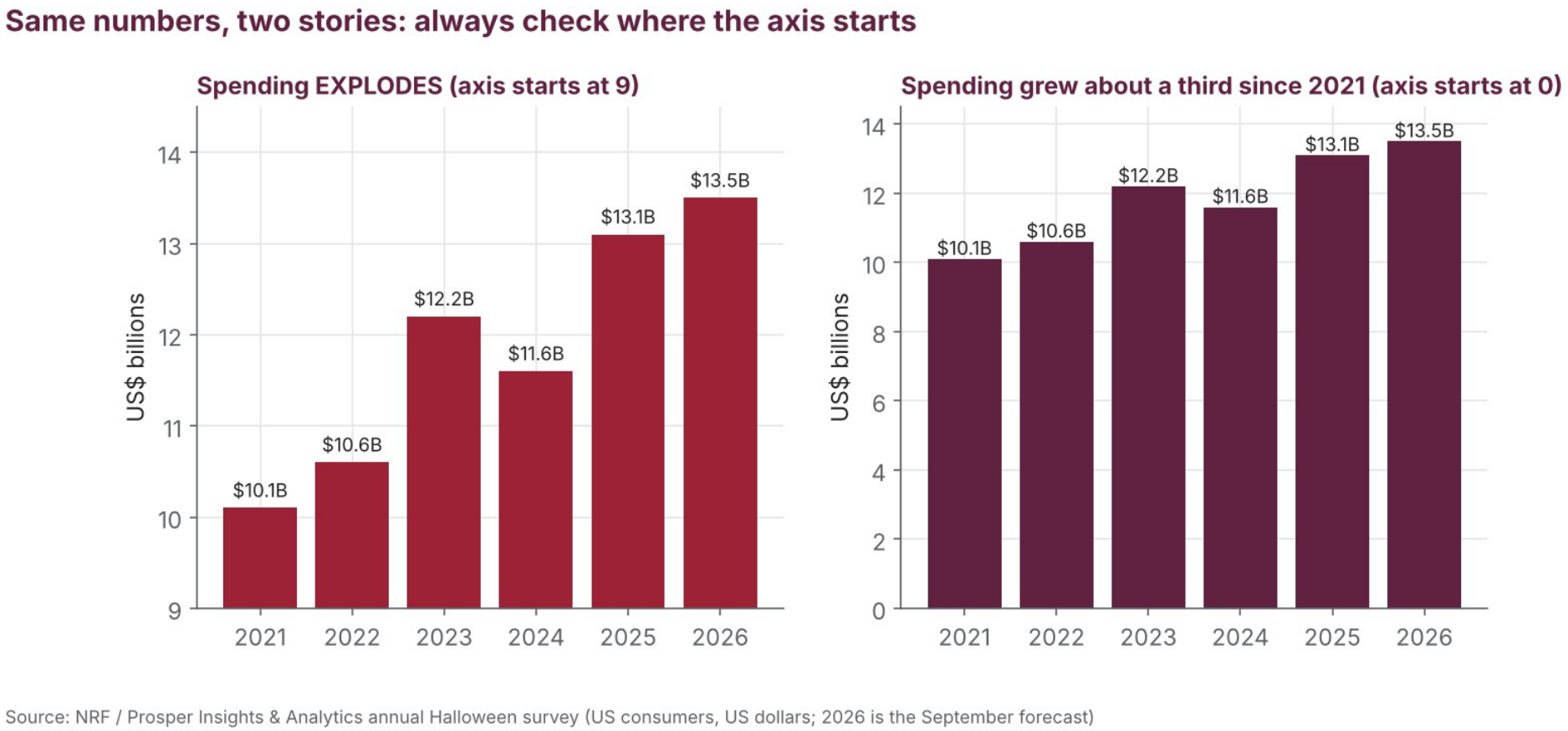
Read across a row for a lane's season, down a column for who is busy this month. October: costumes, pumpkin spice, snow tires. December: gifts and candy. February: valentines. May to July: sunscreen. August: back to school.

THE LIMIT

Monthly averages blur the exact week; season maths finds the week.

Source: Google Trends, Canada, web search, weekly, Oct 24, 2021 to Oct 25, 2026; relative interest 0 to 100 per lane; read Oct 29, 2026

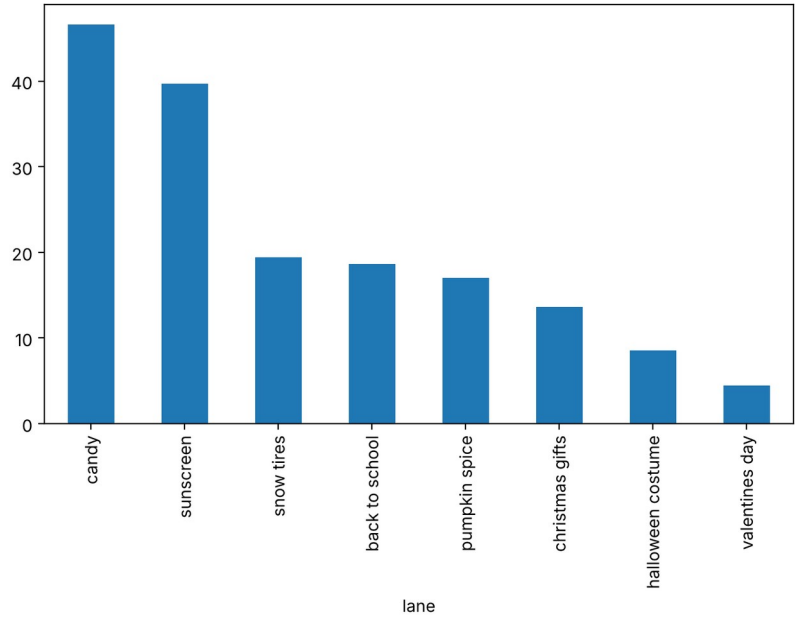
Start bars at zero: an axis starting at 9 turns 34% growth into an explosion



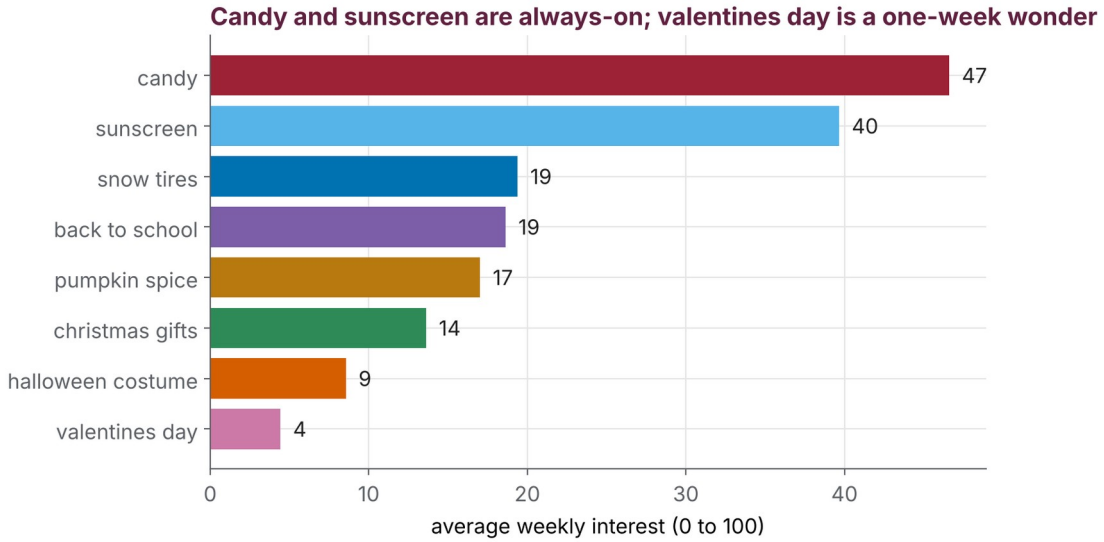
Source: NRF / Prosper Insights & Analytics annual Halloween consumer survey (nrf.com Halloween Data Center); US consumers, US dollars; 2026 is the September forecast

Same numbers, two charts: the second says what it found

BEFORE: DEFAULTS



AFTER: HOUSE STYLE



Source: Google Trends, Canada, web search, weekly, Oct 2021 to Oct 2026 (relative interest, 0 to 100 per lane)

1 Title states the finding

2 Labels you can read

3 One colour per lane

4 The number on the bar

5 A source line

Source: Google Trends, Canada, web search, weekly, Oct 24, 2021 to Oct 25, 2026; relative interest 0 to 100 per lane; read Oct 29, 2026

Four rules for every chart you make today

1

Bars start at zero

a truncated axis turns a third into an explosion

2

The title states the finding

not the topic: 'Costume season is 3 to 4 weeks', not 'Costume searches'

3

A source line, always

where the numbers came from, and when

4

Say the limit out loud

relative interest, not sales · Canada-wide, not Nova Scotia · US dollars · a survey

A chart without a limit is an ad.

04

Season maths: the three numbers every pitch needs

SEASON MATHS

Peak week, stock-up week, season length: a function with two while loops

```
THRESHOLD = 0.5

def season(one_year):
    values = list(one_year["interest"])
    weeks = list(one_year["week"])
    peak_i = values.index(max(values))        # the busiest week
    cut = THRESHOLD * values[peak_i]          # the bar to clear
    start = peak_i
    while start > 0 and values[start - 1] >= cut:
        start = start - 1                     # walk back while busy
    end = peak_i
    while end < len(values) - 1 and values[end + 1] >= cut:
        end = end + 1                         # walk forward the same way
    return {"peak_week": weeks[peak_i].date(), "stock_up_week": weeks[start].date(),
            "season_weeks": end - start + 1}

r = season(one_year)      # halloween costume, 2025
print("stock up", r["stock_up_week"], "· peak", r["peak_week"], "·", r["season_weeks"], "weeks")
```

```
stock up 2025-10-12 · peak 2025-10-26 · 3 weeks
```

peak week

the week with the highest score

stock-up week

walking backwards from the peak, the first week still at or above 50% of it. Shelves full by then.

season length

the stretch of weeks around the peak that stay above the bar

✏️ **THRESHOLD = 0.5** is a choice you made. Change it, the table changes; say so in your pitch.

Source: Google Trends, Canada, web search, weekly, Oct 24, 2021 to Oct 25, 2026; relative interest 0 to 100 per lane; read Oct 29, 2026

SEASON MATHS · EVERY LANE

Same every year? The shortest and longest season for every lane

Lane	Store	Shortest	Longest	Average	Stock-up weeks (2022 to 2025)
halloween costume	party store	3	4	3.5	Oct 9 · Oct 8 · Oct 6 · Oct 12
candy	convenience store	12	16	14	Sep 11 · Sep 17 · Oct 6 · Oct 5
pumpkin spice	café	8	10	9	Aug 21 · Aug 20 · Aug 18 · Aug 17
christmas gifts	gift shop	6	7	6.2	Nov 13 · Nov 12 · Nov 10 · Nov 16
valentines day	florist / card shop	1	2	1.2	Feb 6 · Feb 12 · Feb 11 · Feb 9
sunscreen	pharmacy	15	23	18.5	Apr 24 · Apr 9 · Mar 31 · Mar 9
snow tires	garage	2	11	7.5	Oct 16 · Sep 24 · Sep 29 · Nov 2
back to school	stationery store	5	5	5	Aug 7 · Aug 6 · Aug 4 · Aug 3

Season length in weeks at the 50% rule, full years only (2021 and 2026 are partial). Stock-up week = first week of the busy stretch.

Source: Google Trends, Canada, web search, weekly, Oct 24, 2021 to Oct 25, 2026; relative interest 0 to 100 per lane; read Oct 29, 2026

05

Pitch the season

Teams of 2 or 3. One lane, one store, one decision backed by charts

halloween costume party store	**candy** convenience store	**pumpkin spice** café	**christmas gifts** gift shop
valentines day florist / card shop	**sunscreen** pharmacy	**snow tires** garage	**back to school** stationery store

THE DECISION

Stock date	Ad start	Markdown date	How to check
shelves full by the stock-up week	a week or two before	after the last busy week	real sales next year, not searches

Chart rule 2 or 3 charts from section 6; at least one box plot, heatmap, histogram or scatter; every chart with a title that states the finding, a source, and one honest limit said out loud.

Title + 3 slides, emailed by 12:10

YOUR SLIDES

0 — Title

your lane, your store, your team's names

1 — What we did

one cell explained line by line (5a or 6a is a good pick)

2 — What the data proves

2 or 3 charts, at least one box plot / heatmap / histogram / scatter, and one honest limit

3 — The decision, and how we used AI

stock date, ad start, markdown date, how you'd check it; what the AI did and how you checked it

AI rules (Day 2): no personal information · run it yourself · the AI can be wrong.

SEND ONE OF THESE

A PowerPoint

your 4 slides, charts pasted from the Files panel

A Google Slides link

shared so Gavin can open it

An AI-built HTML dashboard

attached as an .html file. Encouraged!

Want an AI-built dashboard? Dashboard Skill.md is on the download page. Paste your season table or month table from step 6f, not the raw data.

16 points: one row per slide, plus delivery

Criterion	4	3	2	1
What we did (slide 1): explain one cell	Walks through one cell line by line and says why each line is needed	Explains what it does, with small gaps	Reads the code aloud without explaining it	Can't explain it
What the data proves (slide 2): charts	A clear claim, 2 or 3 readable charts (at least one box plot, heatmap, histogram or scatter) from real output, and one honest limit	Claim + charts, but bar or line only, or no limit	Claim with a weak or unlabelled chart	Opinion only, no chart
The seasonal decision (slide 3)	Stock date, ad start and markdown date that follow from the charts, plus how to check it with real sales	Dates given, weakly tied to the charts, or no way to check	Vague ("stock up in the fall")	Missing
Delivery, time and use of AI	Everyone speaks, within 3 min; says what the AI did and how they checked it	Everyone speaks; slightly over time or thin on checking	One speaker dominates, or no AI checking	Unprepared or well over time

Eight teams, three minutes each, in season order

1 back to school

stationery store

2 halloween costume

party store

3 candy

convenience store

4 pumpkin spice

café

5 christmas gifts

gift shop

6 valentines day

florist / card shop

7 snow tires

garage

8 sunscreen

pharmacy

3 minutes each: everyone on the team speaks; the decision slide gets the most time.

We built a program that analyzes and visualizes data to support a business decision.

Python lets us refine insights and prove a decision.

Next Friday: how generative AI writes, and you design an AI assistant.